

Học Sâu (Deep Learning)

Chương 7: Lập trình Chuyên sâu với Keras (A Deep Dive on Keras)

Đại học Đông Á

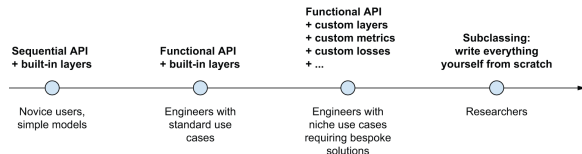
August 14, 2026

Nội dung chương

1. Triết lý Thiết kế của Keras
2. Ba phương pháp Xây dựng Mô hình
3. Tùy biến Vòng lặp Huấn luyện (Customizing)
4. Callbacks: Tự động hóa quá trình Huấn luyện
5. Giám sát với TensorBoard
6. Tổng kết

Từng bước phơi bày sự phức tạp (Progressive Disclosure)

- **Nguyên tắc thiết kế Keras:** Làm cho mọi thứ trở nên dễ bắt đầu, nhưng vẫn hoàn toàn khả thi để xử lý các bài toán siêu phức tạp.
- Không có một "con đường duy nhất". Keras cung cấp một **Phổ quy trình (Spectrum of workflows)**.
- Các trường hợp đơn giản có thể được giải quyết bằng vài dòng mã, trong khi các công việc phức tạp có thể yêu cầu viết mã chi tiết.



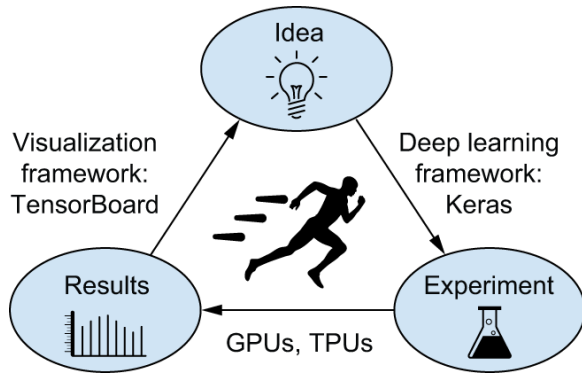
Hình 7.1: Từ Sequential đến Model Subclassing

Spectrum of workflows (Phổ quy trình)

- Keras được thiết kế giống như triết lý của Python: Đa mô hình (multi-paradigm).
- **Người mới (Beginner):** Sử dụng Keras như Scikit-learn (dùng `model.fit()`, `Sequential`).
- **Nhà nghiên cứu (Researcher):** Sử dụng Keras như NumPy/TensorFlow lõi (viết tay mọi thứ từ đầu bằng GradientTape).
- Đặc biệt, các API này **có thể giao tiếp trơn tru với nhau**. Một mô hình `Sequential` có thể được lồng vào một mô hình `Subclassing`.

Vòng lặp Tiến bộ (The Loop of Progress)

- Khoa học là quá trình thử nghiệm lặp đi lặp lại (Iterative Process).
- **Idea:** Khởi tạo ý tưởng.
- **Experiment:** Cài đặt mã nguồn.
- **Evaluate:** Đánh giá kết quả → Rút ra bài học và tạo Idea mới.
- Keras tối ưu hóa trải nghiệm người dùng (UX) để thu ngắn tối đa thời gian lặp trình.



Hình 7.2: Vòng lặp phát triển Mô hình ML

Phương pháp 1: Sequential API

- Cấu trúc dữ liệu dạng **Danh sách (Python List)**.
- Đơn giản nhất, dễ tiếp cận nhất. Các lớp được xếp chồng tuyến tính (lớp sau nối tiếp lớp trước).
- **Hạn chế:** Chỉ áp dụng được cho mô hình 1 đầu vào, 1 đầu ra.
- Không thể áp dụng cho: Mô hình đa đầu vào (Multi-input), đa đầu ra (Multi-output), hay cấu trúc phân nhánh.

Sequential API: Mã nguồn cơ bản

Khởi tạo mô hình Sequential dưới dạng một danh sách (list):

```
1 import keras
2 from keras import layers
3
4 model = keras.Sequential(
5     [
6         layers.Dense(64, activation="relu"),
7         layers.Dense(10, activation="softmax"),
8     ]
9 )
```

Hoặc xây dựng dần dần (incrementally) bằng hàm add():

```
1 model = keras.Sequential()
2 model.add(layers.Dense(64, activation="relu"))
3 model.add(layers.Dense(10, activation="softmax"))
```

Sequential API: Trọng số (Weights) và Shape

Các lớp trong Keras chỉ tạo trọng số (build) khi biết kích thước đầu vào (input shape):

```
1 >>> model.weights # Before build
2 []
3 >>> model.build(input_shape=(None, 3))
4 >>> model.weights # Weights created
5 [<Variable shape=(3, 64) ...>, <Variable shape=(64,) ...>]
```

In bảng tóm tắt cấu trúc:

```
1 >>> model.summary()
2 Model: "sequential_1"
3 ...
```


Sequential API: Khai báo Input Shape trước

Sử dụng đối tượng `Input` để khai báo đầu vào, cho phép mô hình được build tự động ngay từ đầu:

```
1 model = keras.Sequential()
2 model.add(keras.Input(shape=(3,)))
3 model.add(layers.Dense(64, activation="relu"))
4
5 # Model is built automatically, we can call summary now
6 model.summary()
```

Điều này đặc biệt hữu ích khi thiết kế các mạng CNN sâu.

Phương pháp 2: Functional API

- Coi các layer là các **Hàm (Functions)**. Nhận Tensor và trả về Tensor.
- Cung cấp giải pháp linh hoạt hơn nhiều, cho phép xây dựng mạng **Đồ thị đa hướng (Directed Acyclic Graph)**.
- API phổ biến nhất trong thực tế. "Cảm giác như chơi lắp ráp LEGO".
- Cho phép định nghĩa nhiều luồng vào ra, chia nhánh đồ thị.

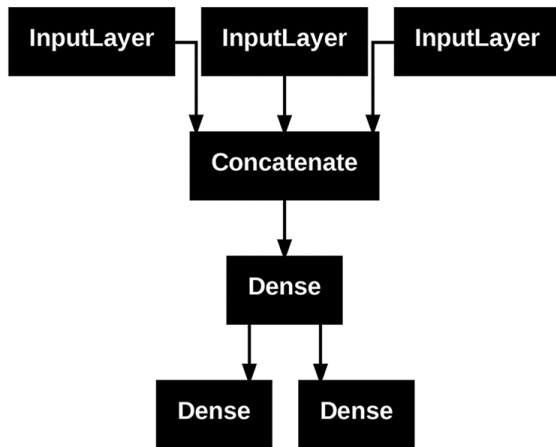
Functional API: Các bước cơ bản

```
1 # 1. Define Input
2 inputs = keras.Input(shape=(3,), name="my_input")
3
4 # 2. Build graph by calling layers like functions
5 features = layers.Dense(64, activation="relu")(inputs)
6
7 # 3. Create outputs
8 outputs = layers.Dense(10, activation="softmax")(features)
9
10 # 4. Initialize Model with inputs and outputs
11 model = keras.Model(inputs=inputs, outputs=outputs, name="
    my_functional_model")
```

Đối tượng `inputs` là một **Symbolic Tensor** (Tensor ký hiệu), chỉ chứa thông tin shape và dtype, không chứa dữ liệu thực.

Functional API: Ví dụ Mạng phân loại vé

- Hệ thống Ticket Classifier.
- **3 Đầu vào (Inputs):**
 - Tiêu đề (title)
 - Nội dung (text_body)
 - Thẻ (tags)
- **2 Đầu ra (Outputs):**
 - Mức độ ưu tiên (priority: 0 đến 1)
 - Phòng ban xử lý (department)



Hình 7.3: Sơ đồ mạng đa luồng

Functional API: Cài đặt hệ thống phân loại vé

```
1 vocabulary_size = 10000
2 num_tags = 100
3 num_departments = 4
4
5 # Inputs
6 title = keras.Input(shape=(vocabulary_size,), name="title")
7 text_body = keras.Input(shape=(vocabulary_size,), name="text_body")
8 tags = keras.Input(shape=(num_tags,), name="tags")
9
10 # Concatenate features
11 features = layers.Concatenate()([title, text_body, tags])
12 features = layers.Dense(64, activation="relu", name="dense_features")(features)
13
14 # Outputs
15 priority = layers.Dense(1, activation="sigmoid", name="priority")(features)
16 department = layers.Dense(num_departments, activation="softmax", name="department")(
    features)
17
18 # Initialize Model
19 model = keras.Model(inputs=[title, text_body, tags], outputs=[priority, department])
```

Functional API: Huấn luyện với Multi-Input/Output

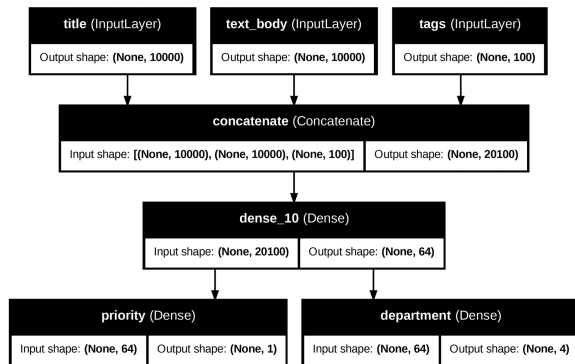
Khi dùng `fit()`, bạn truyền Dữ liệu dạng List hoặc Dictionary:

```
1 model.compile(  
2     optimizer="adam",  
3     loss={  
4         "priority": "mean_squared_error",  
5         "department": "sparse_categorical_crossentropy",  
6     },  
7     metrics={"priority": ["mae"], "department": ["accuracy"]}  
8 )  
9  
10 model.fit(  
11     {"title": title_data, "text_body": text_body_data, "tags": tags_data},  
12     {"priority": priority_data, "department": department_data},  
13     epochs=1  
14 )
```

Sử dụng tên 'name' của Input/Output (Dictionary mode) thường dễ bảo trì hơn so với truyền mảng (List mode).

Functional API: Luồng dữ liệu và Hình dáng (Shapes)

- Có thể vẽ sơ đồ mạng bằng tiện ích `plot_model`.
- Tham số `show_shapes=True` in ra cả kích thước.
- `TensorShape (None, 10000)`: `None` đại diện cho kích thước Lô (Batch size) bất kì.
- Kiểm tra kích thước ở Design Time giúp loại bỏ các lỗi ma trận kích thước lệch pha (Shape Mismatch).



Hình 7.4: Đồ thị truyền tải Shapes